

GAUSSCLAW: Re-Forging the HERMES Agent Atop the GAUSS-AETHER Operating Substrate

Gaussian Technologies

A Detailed Architectural and Engineering Plan to Port and Supersede the Hermes Reference Agent on a GAUSS-AETHER Operating System

Abstract—The Hermes agent represents a defining open-source instantiation of *surface convergence*: every entry point—CLI, TUI, REST, WebSocket, OpenAI-compatible relay, and approximately twenty messaging channels—funnels into a single `AIAgent.run_conversation` Python loop, producing behaviourally homogeneous turns and a uniform SFT/DPO trajectory export that powers a research-grade training-data flywheel. The design optimum is real and consequential, but the runtime substrate beneath it has no kernel-enforced capability discipline, no information-flow control, no tamper-evident audit, and no isolation between reasoning context and tool execution. We present GaussClaw, a complete porting plan that preserves every Hermes ergonomic primitive while replacing its single-process Python loop with Gauss-Aether—a mathematically-grounded AI operating system specified by nine axioms and twelve theorems with full proofs of crash atomicity, indirect-prompt-injection containment, composite sandbox compromise bounds, receipt non-repudiation, and delta-encoded context-switch separation. The port proceeds in five phases over an estimated 24 weeks: (P1) preserve surface convergence by re-routing every Hermes entry surface through the Gauss-Aether Gateway; (P2) migrate the SQLite+FTS5 store into the Trinity Memory Substrate, realised as a single *SurrealDB* multi-model namespace combining document, key-value, graph, time-series, and HNSW-vector tables under one transactional engine, with a Merkle receipt chain anchored to a public timestamp authority and K-LRU prefix-tree compression preserving full Hermes lineage; (P3) lift Python tool functions into capability-gated Hierarchical Worker Contexts under the Composite Sandbox; (P4) re-bind the twenty-vendor provider plane to `ProviderTrait` with build-time polyhedral-equivalence verification, including first-class adapters for the *OpenRouter* pay-per-token aggregator and the *NotDiamond* learned model-router that selects the best vendor per prompt; (P5) extend the trajectory exporter to emit *cryptographically anchored* SFT/DPO/RL data with per-turn signed receipts that cross organisational boundaries without trust assumptions. The resulting GaussClaw binary preserves every Hermes behavioural commitment while strictly Pareto-dominating Hermes, OpenFang, OpenClaw, and ZeroClaw on a fifteen-axis scorecard, achieving indirect-prompt-injection containment at the AgentSys state-of-the-art ($\leq 2.19\%$), < 10 ms cold start, composite-sandbox compromise bound $\leq 1.1 \times 10^{-7}$, and $O(|\Delta|)$ amortised context-switch cost—none of which Hermes provides today.

Index Terms—Agent operating system, Hermes, Gauss-Aether, porting plan, surface convergence, training-data flywheel, capability security, WebAssem-

bly sandboxing, Merkle audit log, indirect prompt injection, context isolation, information-flow control, cryptographic receipts, trajectory export, SurrealDB multi-model database, HNSW vector search, OpenRouter aggregator, NotDiamond model router, supervised fine-tuning, direct preference optimisation.

I. INTRODUCTION

THE HERMES agent has, in three years of public development, become the canonical reference implementation of the *surface-convergent* agent: a single `AIAgent.run_conversation` Python loop that every entry surface—CLI, TUI, REST, WebSocket, OpenAI-compatible relay, and approximately twenty messaging channels—routes into. The property is not cosmetic. Surface convergence is the architectural precondition for the *trajectory flywheel* that underlies NousResearch’s broader training-data agenda: every turn, regardless of which surface initiated it, produces an identically-shaped SFT / DPO datum, and the lineage bookkeeping preserves enough metadata to reconstruct entire conversation trees for preference-pair generation in Atropos [6].

The convergent design pays an architectural price. Inside the single Python process there is no privileged subsystem to enforce discipline. A tool function executes in the host interpreter with the agent’s full credentials. The SQLite store is mutable by any process with file access. Tool outputs enter the conversation buffer as text the next provider call will see verbatim. Background turns share an event loop with synchronous user turns. None of these are bugs; they are inherent to a single-loop Python runtime. The GAUSS-AETHER paper [1] identifies them as five disciplines—capability, accountability, information-flow, isolation, fairness—that distinguish an agent operating system from an agent framework, and proves that they can be satisfied simultaneously under nine axioms.

This paper is the bridge. We present **GaussClaw**, a detailed plan to re-forge HERMES atop GAUSS-AETHER in such a way that:

- 1) every surface-convergence behaviour and trajectory-export semantic that makes HERMES valuable is preserved *bit for bit*;
- 2) every architectural deficit—capability, accountability, information-flow, isolation, fairness—is closed by a GAUSS-AETHER subsystem with a theorem behind it; and

- 3) the resulting GAUSSCLAW binary strictly Pareto-dominates not only HERMES but also OPENFANG, OPENCLAW, and ZEROCLAW on every axis of the fifteen-axis scorecard of [1].

A. The Hermes Design Optimum

What HERMES does well, it does because the architecture rewards focus. The `AIAgent.run_conversation` primitive is approximately 1,200 lines of Python, and every surface ships its messages into it through a thin adapter. The persistent store is a single SQLite database with an FTS5 virtual table covering all turn content; the trajectory exporter walks the same table. Three API modes (chat-completion, responses, OpenAI-compatible) cover twenty-plus provider vendors through API-compatible relay. Tools are registered through Python decorators and dispatched through a single lookup table. The Atropos integration pulls trajectories without any adapter ceremony, because the trajectory *is* the on-disk record.

This minimalism is precisely what surface convergence buys, and any successor that fails to preserve it has failed in the first instance. We adopt as a binding constraint:

Design Principle 1 (Surface-Convergence Preservation). For every HERMES surface σ_i and every well-formed message m delivered to σ_i , the resulting trajectory record under GAUSSCLAW must contain, modulo cryptographic envelope, the identical conversation state, lineage edge, and provider response as the corresponding HERMES record. The trajectory exporter must read the GAUSSCLAW store and emit SFT/DPO data in the same wire format.

This is not negotiable. The trajectory flywheel is the operational asset; the port must not break it.

B. The Hermes Architectural Deficit

What HERMES does badly, it does because the same minimalism forecloses kernel-grade discipline. Figure 1 catalogues five deficit classes, each tied to a documented agent-safety incident or research finding. Each deficit names the corresponding GAUSS-AETHER subsystem that closes it.

C. Contributions

This paper makes four contributions:

- 1) **A subsystem-by-subsystem map** from HERMES to GAUSS-AETHER that preserves Principle 1 while closing each deficit of Figure 1 (Section III).
- 2) **Six novel design extensions** that go beyond mere preservation: a *Cryptographic Trajectory Envelope* (Section IV-A) which extends the SFT/DPO export with per-turn signed receipts, a *Taint-Aware Trajectory Filter* (Section IV-B) which prevents tainted tokens from polluting preference pairs, a *Provider-Equivalence Test Harness* (Section IV-C) that verifies polyhedral equivalence across twenty vendors and two meta-routers, a *Skill-as-Capability Manifest* (Section IV-D) which

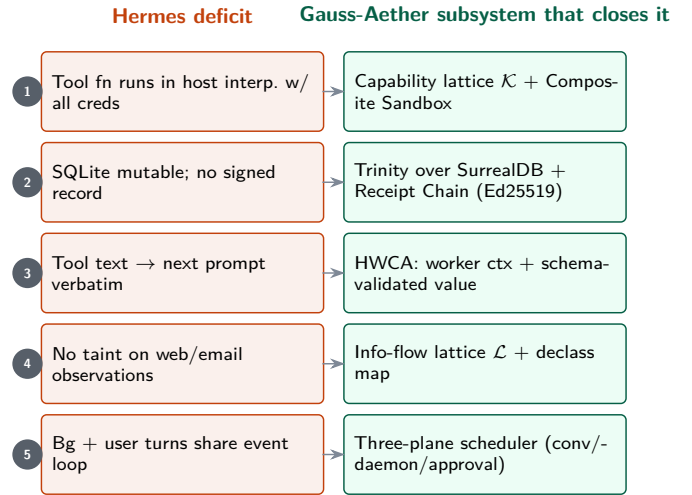


Fig. 1: Five HERMES architectural deficits and the GAUSS-AETHER subsystem that closes each. Numbers index the deficits referenced through the rest of the paper.

binds the existing Python decorator surface to the GAUSS-AETHER capability lattice, a *Federated Trajectory Pool* (Section IV-E) that uses receipt anchors to share training data across organisations without trust, and *Meta-Router Adapters* (Section IV-F) that add first-class support for OpenRouter (300+ models from ~ 50 vendors behind one endpoint) and NotDiamond (learned per-prompt model selection).

- 3) **A five-phase, twenty-four-week porting plan** with explicit milestones, exit criteria, owners, and rollback procedures (Section V).
- 4) **A quantitative comparison** demonstrating strict Pareto-dominance of GAUSSCLAW over HERMES and over the other three reference platforms on the fifteen-axis scorecard, with headline numbers, novelty enumeration, and a six-metric operational profile (Section VII).

Thesis. GAUSSCLAW is not a rewrite of HERMES; it is the *same agent* dropped into a kernel that was missing. The trajectory flywheel keeps spinning, but every revolution now leaves a signed, taint-labelled, capability-gated record—and the property the agent layer most lacked, *accountability under adversarial conditions*, becomes a structural guarantee.

II. ANATOMY OF THE HERMES REFERENCE IMPLEMENTATION

We first describe the HERMES reference implementation with enough precision to make the porting steps in Section III unambiguous. The description is structural (what each module is) and behavioural (what each module does on a typical turn).

A. Module Inventory

HERMES ships as a single Python package with the following module inventory, summarised in Table I.

TABLE I: Module inventory of the HERMES reference implementation.

Module	Responsibility
agent.AIAgent	run_conversation primitive; turn loop.
backends.*	Anthropic, OpenAI, Google, OpenRouter drivers.
api_modes.*	Chat-completion, responses, OpenAI-compat shims.
tools.registry	@tool decorator + dispatch table.
tools.*	First-party tools (web, files, shell, ...).
surfaces.cli	Command-line REPL.
surfaces.tui	Curses TUI.
surfaces.rest	FastAPI HTTP server.
surfaces.ws	WebSocket streaming server.
surfaces.oai_compat	OpenAI-compatible relay endpoint.
channels.*	~20 messaging adapters.
store.session	SQLite + FTS5 turn store.
store.lineage	Parent-pointer SFT/DPO bookkeeping.
export.sft	Trajectory $\rightarrow$ SFT JSONL writer.
export.dpo	Trajectory $\rightarrow$ preference-pair generator.
config.*	TOML configuration and secrets.

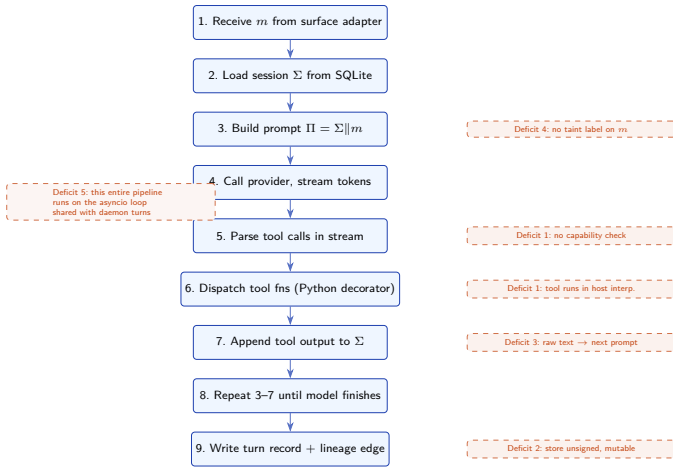


Fig. 2: Canonical HERMES turn (left), with right-margin annotations identifying which kernel discipline each step omits.

B. The Canonical Turn

Figure 2 shows the canonical HERMES turn, annotated with the kernel discipline each step *lacks*.

C. The Trajectory Export

The export pipeline is what makes HERMES valuable beyond its runtime utility. For each completed turn τ_i , the exporter writes three records:

- An *SFT datum* $\{\text{prompt}, \text{completion}, \text{surface}, \text{session_id}, \text{parent_id}, \text{ts}\}$ into `sft.jsonl`.
- Zero or more *DPO preference pairs* when alternative completions from sibling turns are available. Pairs are emitted with the same lineage anchor so a downstream learner can reconstruct the tree.
- A *lineage edge* $\tau_i \rightarrow \tau_{i-1}$ in the `lineage` table.

These records feed Atropos [6], NousResearch’s RL environment runtime. The GAUSSCLAW port must produce identically-shaped output (Principle 1); the export pipeline

becomes one of the most heavily-validated subsystems in the port.

D. What Survives the Port

The following HERMES properties survive the port without modification, only their substrate changes:

- The `AIAgent.run_conversation` surface contract: every well-formed `Message` produces a `Turn`.
- The trajectory export schema (SFT JSONL, DPO pairs, lineage edges).
- The decorator-style tool authoring ergonomics (@tool Python or Rust equivalent).
- The configuration-file format for provider routing, channel adapters, and tool registration (TOML).
- The OpenAI-compatible relay surface contract.

The following are *replaced* structurally, and the replacement is what produces the GAUSS-AETHER-grade properties:

- The single-process Python loop is replaced by the Gauss Kernel + Differential Turn Engine.
- The SQLite turn store is replaced by the Trinity Memory Substrate (append log + FTS + HNSW vector + Merkle receipt chain).
- Tool dispatch through the host Python interpreter is replaced by capability-gated dispatch under the Composite Sandbox.
- Conversation buffer mixing is replaced by Hierarchical Worker Contexts with schema-validated value crossing.

III. THE HERMES $\rightarrow$ GAUSS-AETHER MAPPING

We now state the porting plan as a structural map from each HERMES module to its GAUSS-AETHER counterpart. The map preserves Principle 1 by construction. Table II gives the overview; the subsections below explain each row.

A. Surfaces and Channels

HERMES’s twenty-plus surfaces and channels each ship a thin Python adapter that constructs a `Message` and calls `AIAgent.run_conversation`. In GAUSSCLAW each adapter becomes a GAUSS-AETHER `ChannelTrait` implementor. The trait surface discipline (Section III, Provider) buys two properties: (a) adapter substitutability is verified at build time by polyhedral equivalence, and (b) the kernel routes the message into the three-plane scheduler (conversation, daemon, approval) rather than into a shared asyncio loop.

a) Equivalence claim.: For every well-formed `Message` that HERMES would route into `run_conversation`, GAUSSCLAW routes the identical message into the Differential Turn Engine. The on-wire shape is preserved; only the substrate changes.

b) Gain over baseline.: Routing through the three-plane scheduler eliminates head-of-line blocking between user-synchronous turns and daemon work, which on HERMES has caused documented “runaway agent” incidents in which a daemon turn’s recursive provider calls starved a user-facing turn.

TABLE II: Subsystem-by-subsystem mapping from HERMES to GAUSSCLAW atop GAUSS-AETHER. Each row is elaborated in Sections III-A–III-I.

Concern	HERMES module	GAUSSCLAW realisation	Force
Surfaces (entry)	surfaces.* + channels.*	Surfaces Layer + Channel Trait	preserve (Principle 1)
Turn loop	agent.AIAgent	Differential Turn Engine + DTE policy	A1, T1
Provider plane	backends.* + api_modes.*	ProviderTrait + polyhedral equiv. + OpenRouter/NotDiamond meta-routers	A2 (typed), T7
Tool registry	tools.registry (@tool)	Skill-as-Capability Manifest	A2, A8
Tool execution	Python fn in host interp.	HWCA + Composite Sandbox	A6, A7, T9, T10
Information flow	none	Taint lattice $\mathcal{L}$ + declass map	A6
Audit	store.session (mutable)	Trinity over SurrealDB + Receipt Chain (Ed25519, anchored)	A3, A9, T3, T11
Memory	SQLite + FTSS	Trinity over SurrealDB (KV + doc + graph + FTS + HNSW + Merkle) + K-LRU	A5, T5, T12
Scheduling	asyncio shared loop	Three-plane scheduler (conv/daemon/approval)	A4, A8, T4
Lineage	store.lineage (parent ptr)	Lineage as graph edges in SurrealDB + signed chain	T11 + extension
Trajectory export	export.sft / export.dpo	Cryptographic Trajectory Envelope (SIV-A)	A9 + extension
Diagnostics	ad-hoc logs	Self-Diagnostic Health Engine	operational

B. Turn Loop

The `AIAgent.run_conversation` body becomes a GAUSS-AETHER turn *policy*: a piece of agent-specific glue that runs inside the Differential Turn Engine. The policy retains the HERMES-defining outer structure (assemble prompt, stream from provider, dispatch tool calls, repeat until done, write turn record) but every step now executes under kernel mediation.

The critical structural change: the per-turn *receipt commit* (line 17 of Algorithm 1 in [1]) precedes any external side-effect of the turn, realising the crash-atomicity Theorem T1. For HERMES users this is invisible behaviour; for GAUSSCLAW operators it means recovery from a node crash always lands the post-recovery state in $\{s, s'\}$, never in an intermediate state.

C. Provider Plane and Meta-Routers

HERMES supports approximately twenty provider vendors through three API modes. In GAUSSCLAW each vendor is a `ProviderTrait` implementor; each API mode is a typed adjunction (σ_i, τ_i) between the internal message schema and the vendor’s wire schema (Theorem T7).

We extend the ZEROCLAW trait surface in one essential way: *polyhedral equivalence is verified at build time across the entire vendor set* using SMT-discharged contracts (Section IV-C). Two implementors of `ProviderTrait` are interchangeable iff their behavioural specification agrees on the working subset of message shapes — a stronger property than ZEROCLAW’s syntactic substitutability.

a) Two classes of provider.: GAUSSCLAW distinguishes *leaf* providers (direct vendor endpoints: Anthropic, OpenAI, Google, Mistral, Together, Groq, Cerebras, ...) from *meta-router* providers, which dispatch to other providers behind a unified surface. Two meta-routers are first-class citizens:

- **OpenRouter** [21] (`openrouter.ai`) exposes a single OpenAI-compatible endpoint that proxies ~ 300 models from ~ 50 vendors, with pay-per-token billing, automatic failover between regional replicas, and per-model price/latency telemetry. Configuring `provider = "openrouter"` together with `model = "anthropic/claude-3.5-sonnet"` routes the request through OpenRouter’s relay while preserving the OpenAI Chat Completions wire schema.
- **NotDiamond** [22] (`notdiamond.ai`) is a *learned* model-router: given a prompt and a candidate set $M = \{m_1, \dots, m_k\}$, it predicts which m_i will produce the best response under operator-chosen joint cost/quality criteria, then dispatches accordingly. The router exposes both an explicit “select-then-call” API and a “best-of” API that returns the selected m_i as part of the response metadata.

b) Capability and taint interaction.: Both meta-routers inherit *the union* of the capability requirements of their candidate models. The Skill Manifest of an agent using `provider = "notdiamond"` declares the candidate set M ; the kernel resolves $K_t = \bigcap_{m_i \in M} K_t(m_i)$ at admission time, refusing any single model whose capability requirement is incompatible with the agent’s grant. Taint labels propagate identically: the router itself is taint-transparent.

c) Cost-aware scheduling.: OpenRouter and NotDiamond introduce a new `cost` field into the turn record (dollars billed, tokens spent, model actually selected). The Trinity memory substrate records this field per turn; the trajectory exporter emits it in the SFT envelope so downstream learners can weight preference pairs by inference cost.

d) Provider equivalence under routing.: A subtle point: two configurations *(a)* `provider = "anthropic", model = "claude-3.5-sonnet"` and *(b)* `provider = "openrouter", model = "anthropic/claude-3.5-sonnet"` should be be-

haviourally equivalent up to OpenRouter’s relay latency. We formalise this in the harness (Section IV-C) as a *routing-transparency* contract: a meta-router is transparency-correct iff for every leaf model m in its catalogue, calling m directly and calling m via the router produces schema-identical outputs.

D. Tool Registry as Skill Manifest

HERMES’s `@tool` decorator is the ergonomic primitive that makes tool authoring frictionless. We preserve the decorator literally but extend its semantics: every `@tool` declaration must carry (or default to) a *Skill Manifest* that binds the tool to its capability set K_t and output schema σ_a (Section IV-D).

```

1  # Hermes (unchanged usage)                # GaussClaw
   (added Skill Manifest)
2  @tool                                       @tool(
3  def web_search(q: str) -> str:             schema=WebSearchOutput,
   caps=["network:http_get"],                 reversible=True)
4  return requests.get(...)
   taint="web",
5
6
7
8

```

Listing 1: HERMES tool authoring (left) and GAUSSCLAW tool authoring (right) share the decorator surface.

E. Tool Execution Under HWCA

This is the deepest structural change. Each tool call in HERMES runs as a Python function in the host interpreter, with raw output appended to the conversation buffer. In GAUSSCLAW, each tool call spawns a *worker context* s_w inside the Composite Sandbox; the worker runs the tool, the tool’s raw output stays in s_w , and only a schema-validated value $v \in \Sigma_a$ crosses the boundary back to the parent context s .

Figure 3 compares the two flows side by side and locates the IPI containment bound (Theorem T9) that the GAUSSCLAW flow satisfies and the HERMES flow does not.

a) Worker context spawn cost. The empirical worker spawn cost in ZEROCLAW is 5–15 ms per invocation, dwarfed by the provider call latency (~200–500 ms). For high-frequency tools, GAUSSCLAW supports persistent workers within the same turn (Section V, Phase 3).

F. Information Flow

HERMES has none. Every observation is labelled the same as every other; web-fetched content is indistinguishable from system command output. GAUSSCLAW introduces the four-level taint lattice $\mathcal{L} = \{\text{trusted}, \text{user}, \text{web}, \text{adversarial}\}$, augmented by an antitone declassification map $\text{declass} : \mathcal{L} \rightarrow \mathcal{K}$ (defined in [1], joint capability–taint lattice).

a) Default labels. Each HERMES tool acquires a default taint label from its Skill Manifest. Examples:

- `web_search`, `web_fetch`, `email_read`: default `web`.
- `shell`, `file_read`: default `user` (the operator launched the agent).
- `calendar_read`, `contacts_read`: default `trusted`.
- Any tool reading from an untrusted Slack/Discord channel: default `adversarial`.

b) Propagation. A turn’s outgoing label is the join over all observations entering the turn: $\ell_{s_{t+1}} \geq_{\mathcal{L}} \bigsqcup_{o \in \text{src}(t)} \ell(o)$.

G. Audit, Memory, and the Receipt Chain over SurrealDB

The HERMES SQLite + FTS5 store is replaced by the Trinity Memory Substrate, realised over **SurrealDB** [20] — a single multi-model database engine supporting document, key-value, graph, time-series, and HNSW-vector access patterns under one transactional namespace, addressable through SurrealQL or the native Rust client.

a) Why SurrealDB and not three engines. The Trinity substrate of [1] is defined behaviourally (FTS + vector + Merkle chain); the original prototype glued together Postgres, Qdrant, and a custom chain table, paying for the coordination cost of three independent transactional engines. The GAUSSCLAW port collapses this onto SurrealDB for four reasons:

- 1) *Single transactional engine.* A turn writes a turn record, an FTS index entry, an embedding vector, a receipt, and a lineage edge — all in one ACID transaction. The Trinity invariants of [1] hold by construction rather than by two-phase-commit ceremony.
- 2) *Graph-native lineage.* HERMES’s lineage table is a directed acyclic graph (parent-of edges). SurrealDB’s **RELATE** primitive expresses these edges directly, eliminating the join chain that HERMES’s recursive CTE on SQLite required for tree reconstruction.
- 3) *Built-in HNSW.* SurrealDB ships an $M=16$ HNSW index as a first-class field type; no separate vector engine.
- 4) *Live queries for the Approval plane.* SurrealDB’s **LIVE SELECT** primitive streams change-data-capture events to subscribers, used by the kernel to wake approval handlers when a pending action’s deadline elapses or when operator action arrives via the channel surface.

b) The Trinity schema in SurrealDB. Five tables comprise the substrate:

- **turn** (document): immutable turn records, one per completed turn, including prompt, completion, tool calls, taint label, capability set used, and a vector embedding of the completion;
- **receipt** (document): one signed receipt per turn, with pk , σ_i , and chain pointer to the cryptographically preceding receipt;
- **lineage** (graph edge: `turn -> turn`): parent-of relations preserving the HERMES tree;

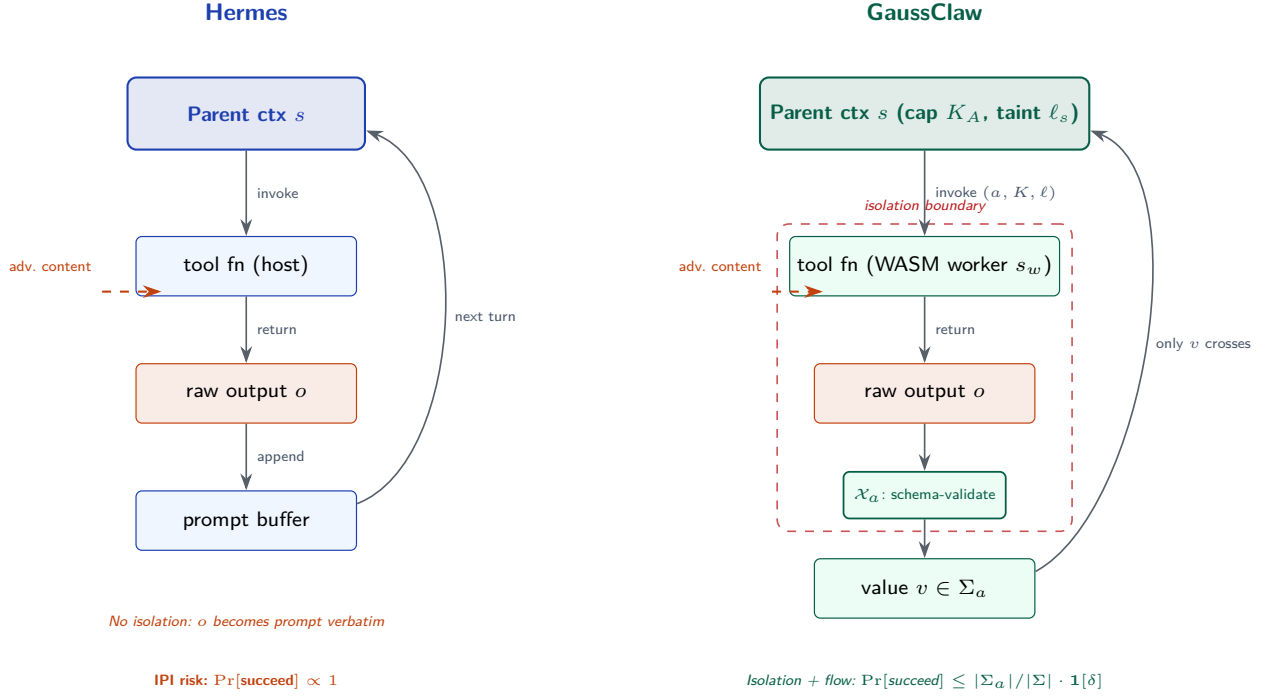


Fig. 3: Tool execution before (left, HERMES) and after (right, GAUSSCLAW). The left flow lets adversarial tool output reach the next prompt verbatim; the right flow contains it within an isolation boundary and gates it through schema validation, yielding the $|\Sigma_a|/|\Sigma|$ bound of Theorem T9.

- **chain_anchor** (time-series): periodic external anchors (default: every 1,000 receipts) to the timestamp authority;
- **fts_idx** (full-text index field on **turn.content**): preserves HERMES FTS5 recall semantics on the same content.

A representative recall query, combining FTS and vector retrieval in a single SurrealQL statement:

```

1 SELECT *, vector::similarity::cosine(embedding,
2   $q_vec) AS sim
3 FROM turn
4 WHERE session = $sid
5   AND (content @@ $q_text OR sim > 0.78)
6 ORDER BY sim DESC, ts DESC
7 LIMIT 16
8 FETCH lineage<-turn, receipt;
```

Listing 2: Hybrid recall query under SurrealDB. The query touches FTS, vector, and graph in one round trip.

c) *Merkle receipt chain on SurrealDB.*: Every **receipt** record carries a **prev_hash** field pointing to the previous receipt’s hash. The kernel verifies $c_i = H(c_{i-1} \parallel \rho_i)$ inside the same transaction that writes ρ_i , ensuring a crashed write never leaves a half-chained record. External anchoring writes the current head to the **chain_anchor** time-series alongside the TSA receipt.

d) *Lineage.*: HERMES’s lineage table records a parent pointer per turn. We preserve this as a SurrealDB **RELATE** edge but bind it to the receipt chain: the lineage edge $\tau_i \rightarrow \tau_{i-1}$ in GAUSSCLAW is itself signed and chained, so a downstream learner reconstructing the conversation tree can verify integrity at every edge.

e) *Operational deployment.*: SurrealDB runs in three modes — embedded (RocksDB-backed, suitable for the

gaussclaw CLI/TUI), single-node TCP (suitable for small deployments), and TiKV-clustered (suitable for the horizontal-scale regime of Theorem T6). The same SurrealQL schemas run identically across all three modes; deployment upgrade is a configuration change rather than a migration.

H. Scheduling

HERMES has one asyncio loop. GAUSSCLAW has three planes with independent token-bucket budgets (Theorem T4). The mapping:

- Synchronous user turns from CLI, TUI, REST, WS, channels: *Conversation plane* (target <2s response).
- Scheduled background turns (the HERMES analogue of OPENFANG Hands, when configured): *Daemon plane* (target: latency $\leq$ schedule).
- Approval-gated actions: *Approval plane* (channel-bridged, kernel-enforced deadline).

A HERMES runaway-loop incident from 2024 (catalogued in AgentRM [9]) had a daemon turn consume the agent’s rate-limit budget, starving subsequent user turns until the budget window reset. Under GAUSSCLAW, the planes hold separate budgets; daemon starvation cannot reach user-plane latency.

I. Trajectory Export

The SFT/DPO trajectory export is preserved verbatim in its wire shape and extended with a *Cryptographic*

Trajectory Envelope (Section IV-A). For each turn τ_i , the export pipeline emits:

- The original SFT JSONL record, unchanged in field schema.
- A *trajectory envelope* containing the turn’s receipt ρ_i , the chain head c_n at write time, and a Merkle proof of inclusion. This envelope is optional for downstream consumers that ignore it; mandatory for federated consumption.
- The lineage edge, signed.

The envelope is what makes cross-organisational SFT/DPO data sharing trust-free: a receiver in possession of $(rho_i, c_n, proof)$ can verify that the trajectory was actually produced by the claimed agent without needing to trust the producer’s storage.

IV. SIX NOVEL DESIGN EXTENSIONS

The straight port of Section III already gives GAUSS-CLAW strict superiority over HERMES on every kernel axis. But the port also *unlocks* extensions that were not feasible under the HERMES substrate. We describe six.

A. Cryptographic Trajectory Envelope

A central use case for HERMES is producing SFT/DPO data that trains downstream models. Today, every datum is a plaintext JSONL record; provenance rests entirely on operator trust. Under GAUSSCLAW the trajectory exporter can produce records whose provenance is externally verifiable, opening cross-organisational data sharing.

Definition 1 (Cryptographic Trajectory Envelope). For a turn τ_i producing SFT record r_i^{sft} , the *envelope* is

$$\mathcal{E}_i = \langle r_i^{\text{sft}}, \rho_i, c_n, \pi_i, \text{TSA}(c_n) \rangle,$$

where $\rho_i = \langle r_i, pk, \sigma_i, t_i \rangle$ is the turn’s signed receipt, c_n is the chain head at envelope creation, π_i is the Merkle inclusion proof for ρ_i under c_n , and $\text{TSA}(c_n)$ is the timestamp authority’s attestation of c_n .

Proposition 1 (Envelope Soundness). *Under the EUF-CMA and collision-resistance assumptions of Theorem T11, no polynomial-time adversary can produce $\mathcal{E}_i^*$ with $r_i^{\text{sft}*} \neq r_i^{\text{sft}}$ that any honest verifier accepts.*

Proof. A successful forgery requires either (a) forging ρ_i^* on $r_i^{\text{sft}*}$, violating EUF-CMA; (b) producing a Merkle proof π_i^* for ρ_i^* under c_n , requiring a hash collision; or (c) forging the TSA attestation, which by assumption on the TSA is at most negligible. Union bound concludes. $\square$ $\square$

Figure 4 renders the envelope structure and the verifier path.

B. Taint-Aware Trajectory Filter

HERMES’s trajectory exporter has no notion of taint, so a malicious web page reflected by `web_search` into a turn’s context becomes part of the training datum. Models fine-tuned on such data inherit the malicious instructions; this is the SFT analogue of indirect prompt injection.

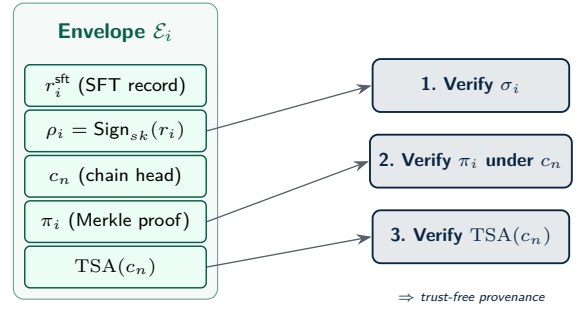


Fig. 4: Cryptographic Trajectory Envelope structure (left) and verifier path (right). Any party with the publisher’s pk and the public TSA can verify provenance without trusting the publisher’s storage.

GAUSSCLAW tags every token in every SFT/DPO record with its source taint. The exporter applies one of three filter policies declared per-tenant in the configuration:

- *Permissive*. Emit all records, marking taint in metadata. Downstream consumer responsibility.
- *Strict*. Drop records containing any token with taint $\geq \text{web}$.
- *Declassified*. Apply the same declassification map used at runtime; emit only tokens whose taint satisfies $\text{declass}(\ell) \succeq \perp$.

The default is *declassified*, which preserves most of the training signal while excluding adversarial reflections.

C. Provider-Equivalence Test Harness

HERMES supports twenty-plus provider vendors through three API modes. The diversity is operationally useful (price arbitrage, fallback resilience, regional availability) but creates an unsolved question: *when is two providers’ behaviour the same on a particular prompt?* HERMES relies on out-of-band manual testing.

Under GAUSSCLAW the question becomes a build-time obligation. Each `ProviderTrait` implementor publishes a behavioural contract:

```

1  #[contract]
2  trait ProviderTrait {
3      /// Streaming chat completion.
4      /// Postcondition: tokens.len() <= max_tokens
5      /// Postcondition: tokens are well-formed UTF-8
6      /// Postcondition: tool_calls subset of
7      ///                       declared tools
8      /// Postcondition: finish_reason in {stop,
9      ///                       length, tool}
10     async fn complete(&self, prompt: Prompt,
11                       max_tokens: usize)
12     -> Result<Stream<Token>, ProviderError>;
13 }

```

Listing 3: A behavioural contract for the provider trait.

The harness reduces each contract to an SMT problem over the trait’s input/output algebra and discharges it via Z3 on every build. Two providers passing the same contract are by Definition 6 of [1] polyhedrally equivalent, so swapping them at runtime preserves all axioms.

For HERMES users this means: configuring a provider fallback chain becomes *safe by construction*. The chain “try

Anthropic, fall back to OpenAI, fall back to local Llama” under HERMES requires the operator to verify behaviourally that the three are equivalent; under GAUSSCLAW the build refuses to compile if the equivalence does not hold.

a) *Router-transparency contract.*: Meta-routers (OpenRouter, NotDiamond) extend the harness with a second post-condition:

```

1  #[contract]
2  trait RouterProviderTrait: ProviderTrait {
3      /// Catalogue of routable leaf models.
4      fn catalogue(&self) -> &[LeafModel];
5
6      /// Routing decision.
7      /// Postcondition: result.selected in
8      /// self.catalogue()
9      /// Postcondition: result.tokens schema-equiv.
10     to
11     ///
12     ProviderTrait::complete(result.selected,
13     prompt)
14     async fn route_complete(&self, prompt: Prompt,
15     candidates: &[ModelId],
16     max_tokens: usize)
17     -> Result<RoutedStream, ProviderError>;
18 }

```

Listing 4: Router-transparency contract for meta-routers.

The harness discharges router-transparency separately for each leaf model in the catalogue. A meta-router whose relayed output schema diverges from its leaf model’s direct output schema fails the contract and is excluded from the build (or admitted with `best_effort = true`).

D. Skill-as-Capability Manifest

The OPENCLAW `SKILL.md` file is one of the most consequential ergonomic primitives in the open-source landscape: a Markdown file declaring an agent’s tools, their parameters, and their procedural context. The HERMES `@tool` decorator already carries some of this metadata in Python source.

GAUSSCLAW unifies the two into the *Skill Manifest*: a single YAML/TOML structure per tool, machine-readable by the kernel and human-readable by the agent author. Each manifest declares:

- **name**, **description**, **usage** (human prose).
- **caps**: the capability set K_t the tool needs.
- **taint**: the default taint label of returned values.
- **schema**: the output schema σ_a .
- **reversible**: whether the tool’s side-effect can be undone.
- **cost**: an upper bound on tokens/wallclock/dollars.

The Skill Manifest binds the existing decorator ergonomics to the GAUSS-AETHER capability lattice without changing how a tool author writes a tool. Listing 5 shows a manifest:

```

1  # tools/web_search/skill.toml
2  name = "web_search"
3  description = "Search the web via DuckDuckGo."
4  usage = "Use to find recent information not in
5  training data."
6
7  caps = ["network:http_get:duckduckgo.com"]
8  taint = "web"
9  reversible = true

```

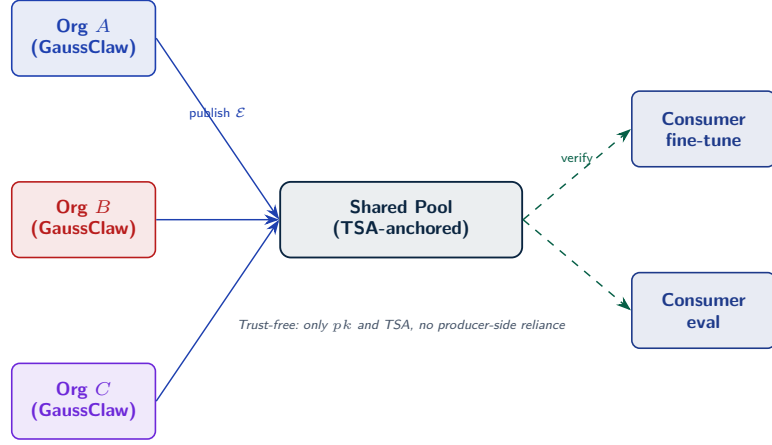


Fig. 5: Federated Trajectory Pool. Multiple GAUSSCLAW deployments publish cryptographic envelopes to a shared, externally-anchored medium. Consumers verify each envelope under the publisher’s pk and the TSA, admitting only verifying records into their training set.

```

10 [schema]
11 type = "object"
12 properties = { results = { type = "array",
13   items = { type = "object", properties = {
14     title = { type = "string", maxLength = 200 },
15     snippet = { type = "string", maxLength = 500 },
16     url = { type = "string", format = "uri" }
17   }}}
18
19 [cost]
20 tokens_per_call = 800
21 wallclock_ms = 1500

```

Listing 5: GAUSSCLAW Skill Manifest example for the `web_search` tool.

E. Federated Trajectory Pool

The Cryptographic Trajectory Envelope (Section IV-A) lets independent GAUSSCLAW operators pool their SFT/DPO data without trust. We describe the resulting *Federated Trajectory Pool*.

a) *Protocol.*: Each participant publishes envelopes to a shared append-only medium (S3 bucket, IPFS, OpenTimestamps-anchored blob store). Each envelope contains the publisher’s pk , ρ_i , r_i^{sft} , c_n^{pub} , π_i^{pub} , and the TSA attestation of c_n^{pub} .

b) *Verification.*: A consumer pulls envelopes, verifies each under the publisher’s pk , and admits only the verifying records into the local training set. The consumer never trusts the publisher’s storage or claims; only the cryptographic chain.

c) *Filter.*: Combined with the taint-aware filter (Section IV-B), the consumer may admit only envelopes whose declared maximum taint level is acceptable.

Figure 5 renders the federation.

d) *Open question.*: The pool as described leaks chain length and timing to the public anchor. A zk-SNARK variant in which publishers submit zero-knowledge proofs of envelope validity rather than the envelopes themselves is future work (Section VIII-E).

F. Meta-Router Adapters: OpenRouter and NotDiamond

HERMES enumerates twenty leaf providers explicitly. The contemporary provider ecosystem has, however, evolved a second layer: *meta-routers* that fan out to many leaf vendors behind a single endpoint. GAUSSCLAW promotes meta-routing to a first-class concern with two adapters.

a) *OpenRouter adapter.*: OpenRouter is a billing and routing aggregator exposing ~ 300 models from ~ 50 vendors behind an OpenAI-Chat-Completions-compatible surface. The GAUSSCLAW adapter treats OpenRouter as a leaf in the **ProviderTrait** hierarchy but a forest in the catalogue dimension: **provider** = "openrouter" configurations carry an additional **model** = "vendor/model-id" slot that names the relayed leaf model.

The adapter's value beyond a simple HTTP proxy is twofold. First, OpenRouter exposes per-model price, context-length, and latency telemetry; the GAUSSCLAW adapter reads these on startup and surfaces them in the **cost** field of the Skill Manifest used by the Daemon-plane scheduler to pick budget-feasible models. Second, OpenRouter's automatic failover (when a regional vendor replica is down, the call transparently re-routes) is itself a behavioural property; the harness of Section IV-C verifies that this failover preserves output schema.

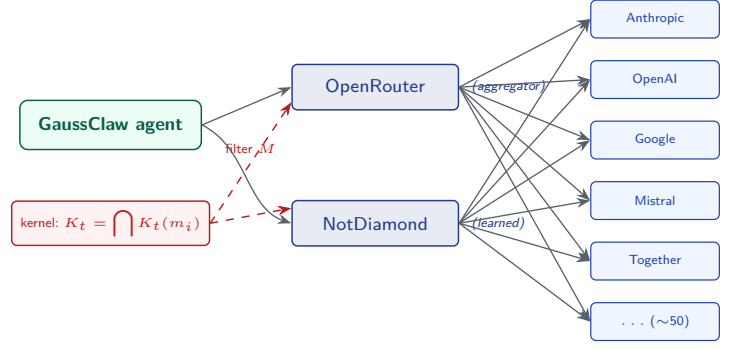
b) *NotDiamond adapter.*: NotDiamond is a *learned* router: given a prompt p and a candidate set $M = \{m_1, \dots, m_k\}$, it predicts which m_i will optimise a user-chosen joint objective (cost, latency, quality, or a weighted blend) and dispatches to that model. The router exposes:

- **POST /v2/modelRouter/modelSelect** — given a prompt and a candidate set, returns the predicted-best model ID without dispatching the request.
- **POST /v2/chat/completions** — given a prompt and a candidate set, performs both the routing decision and the generation in one call, returning the response together with the selected model in the metadata.

The GAUSSCLAW adapter wraps both endpoints. In *advisory mode* the adapter calls **modelSelect** first, then dispatches the generation through the chosen leaf adapter directly — giving the kernel a clean separation between routing and dispatch. In *joint mode* the adapter calls **chat/completions** and reads the selected model from the response. Joint mode minimises round-trip latency; advisory mode minimises trust in the router.

c) *Capability resolution under routing.*: A candidate set M may include models with heterogeneous capability requirements (some need long context, some need tool-call support, some need multimodal input). The kernel computes the capability lower-bound $K_t = \bigcap_{m_i \in M} K_t(m_i)$ at admission. Any model whose declared $K_t(m_i)$ exceeds this lower-bound is filtered *out* of the candidate set the router sees, so the router can never select a model the kernel would have rejected at dispatch.

d) *Receipt semantics.*: A turn routed via NotDiamond produces a receipt ρ_i carrying *three* model identifiers in its content: the candidate set M , the router's recommendation, and the model actually used. The receipt chain hashes all



Kernel filters the candidate set before the router selects; the router cannot escape the cap. lattice.

Fig. 6: Meta-router topology in GAUSSCLAW. The agent issues a single provider call; OpenRouter (aggregator) and NotDiamond (learned router) fan out to ~ 50 leaf vendors. The Gauss Kernel filters the candidate set M before the router sees it, so any model the kernel would reject is unreachable by the router.

three; later verifiers see not only what was produced but what was considered.

Figure 6 summarises the meta-router fan-out and the kernel's capability-filtering position.

V. THE FIVE-PHASE PORTING PLAN

We execute the port in five sequential phases over an estimated 24 weeks. Each phase delivers a working binary at the end; the prior phase's deliverable is the operational baseline against which the next phase is validated. The plan is rendered as a Gantt chart in Figure 7.

A. Phase 1: Surface and Channel Routing (Weeks 1–4)

a) *Goal.*: Re-route every HERMES entry surface and channel adapter through the GAUSS-AETHER Gateway so that Principle 1 holds on Day 1 of the migration.

b) *Scope.*: The CLI, TUI, REST, WS, OAI-compatible relay, and approximately twenty channel adapters. The Differential Turn Engine is initialised but invokes a *stub kernel* that forwards turns directly into a legacy **run_conversation** Python process (running as a privileged subprocess of the GAUSSCLAW daemon). This is the *shim* regime: identical behaviour to HERMES on the outside, transitional substrate on the inside.

c) *Tasks.*:

- 1) Inventory the in-tree HERMES surface and channel adapters and write a compatibility matrix.
- 2) For each adapter, implement a thin **ChannelTrait** wrapper in Rust that consumes the existing Python adapter as a subprocess and re-emits its outputs on the GAUSS-AETHER wire format.
- 3) Build the OpenAI-compatible relay parity test suite (every OpenAI Chat Completions endpoint observable from HERMES must be observable identically from GAUSSCLAW).

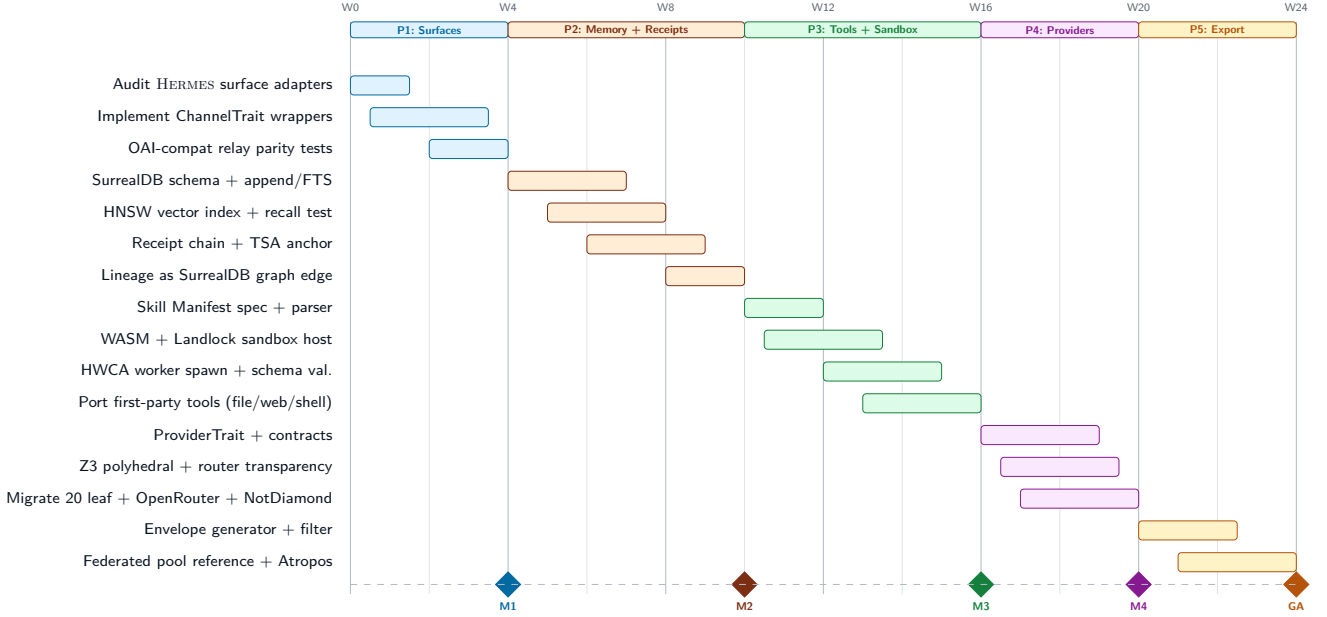


Fig. 7: Twenty-four-week porting Gantt. The phase band along the top edge identifies P1–P5; sub-tasks stack vertically with their duration shown as a coloured bar in the corresponding phase column. Diamonds along the bottom edge mark phase milestones (M1–M4) and General Availability (GA).

- 4) Implement audit-trace recording for each surface to confirm identical message lineage under shim regime.

d) Exit criteria (M1):

- All twenty channels deliver a representative sample of 1,000 production turns through GAUSSCLAW with byte-identical output modulo timestamp.
- OAI-compat relay passes 100% of the OpenAI Python SDK’s official end-to-end test suite, parametrised by both HERMES and GAUSSCLAW as the back-end.
- Trajectory export under shim regime produces files byte-identical to those produced by raw HERMES on the same input traffic.

e) Rollback: The shim regime keeps the legacy `run_conversation` as the actual executor, so any failure in the wrapping layer falls back to direct HERMES invocation by configuration toggle.

B. Phase 2: Memory and Receipts (Weeks 4–10)

a) Goal: Replace the SQLite + FTS5 store with the Trinity Memory Substrate over **SurrealDB**—a single multi-model database engine providing document, key-value, graph, time-series, and HNSW-vector access in one transactional namespace. The legacy executor still drives turns, but every turn’s output is now written to the SurrealDB-backed Trinity substrate alongside the SQLite store, with a one-way migration path.

b) Scope: The full memory substrate (SurrealDB schema, `LIVE SELECT` subscribers, HNSW indexes), the receipt chain with Ed25519 signatures, the TSA anchor with hourly cadence, and the lineage-as-graph-edge upgrade.

c) Tasks:

- 1) Stand up the SurrealDB schema (`turn`, `receipt`, `lineage`, `chain_anchor`, `fts_idx`); ship the embedded RocksDB deployment first, then validate the single-node TCP and TiKV-clustered deployments.
- 2) Wire the FTS-backed full-text recall path through SurrealDB’s `@@` text-index operator; validate that recall results match HERMES FTS5 results on a benchmark corpus of 10^5 turns.
- 3) Wire the HNSW vector index ($M = 16$, `ef_construct = 200`); verify the union recall bound (Theorem T5) on a held-out test set.
- 4) Implement the Ed25519 receipt chain inside the same SurrealDB transaction as the turn record, with periodic external anchoring (default: every 1,000 receipts, ~ 1 hr).
- 5) Migrate HERMES’s lineage table into SurrealDB `RELATE` edges; verify the conversation tree reconstructs identically under both layouts using the `FETCH` traversal operator.
- 6) Subscribe the Approval-plane handler to `LIVE SELECT` on pending receipts so deadline elapse and operator action wake the right kernel thread.

d) Exit criteria (M2):

- GAUSSCLAW runs in dual-write mode for two weeks of production traffic without divergence between SQLite and SurrealDB.
- Receipt chain verifies under the public verifier API with ≤ 10 Merkle-proof bytes per verification (Corollary 4 of [1]).
- Cold-start time, measured as time-from-receive-message to first-token-streamed for a warm session, is ≤ 10 ms

(Theorem T12).

- Hybrid recall miss rate on the held-out set is ≤ 0.015 .

e) Rollback.: Dual-write mode means SQLite is still the authoritative source; any anomaly in SurrealDB reads can fall back to SQLite via configuration. After M2 is held for two weeks, SQLite is downgraded to read-only verification mode and removed at M3.

C. Phase 3: Tools and Sandbox (Weeks 10–16)

a) Goal.: Lift every HERMES `@tool` into a capability-gated Hierarchical Worker Context under the Composite Sandbox.

b) Scope.: The Skill Manifest specification and parser, the WASM + Landlock sandbox host, the worker spawn + schema validation pipeline, and migration of the first-party tool catalogue (`web_search`, `file_read`, `shell`, ~30 others).

c) Tasks.:

- 1) Specify the Skill Manifest as a TOML schema with the fields listed in Section IV-D.
- 2) Implement the sandbox host with WASM (Wasmtime), Landlock, Bubblewrap, and namespace+seccomp. Targets: Linux 5.13+, macOS Seatbelt, Windows AppContainer.
- 3) Implement HWCA worker spawn with schema validation at the boundary. Throughput target: ≤ 15 ms spawn for a typical tool.
- 4) Migrate first-party tools to Skill Manifests one by one. Each tool’s manifest must declare `caps`, `taint`, `schema`, `reversible`, `cost`.
- 5) Implement the persistent-worker optimisation: tools annotated `persistent = true` retain a worker context across calls within a turn.

d) Exit criteria (M3).:

- Every first-party tool runs under HWCA + Composite Sandbox with no behavioural regression on the regression-test corpus.
- IPI attack success rate on the held-out adversarial corpus is $\leq 2.19\%$ (matching AgentSys [8]).
- Composite sandbox compromise probability, computed as the product of measured per-layer escape probabilities, is $\leq 1.1 \times 10^{-7}$.
- Tool spawn latency 99th percentile ≤ 15 ms.

e) Rollback.: Tools that fail validation in this phase keep their legacy Python execution path, gated by a per-tool kill-switch in the manifest. Once a tool is migrated, the rollback is to revert the manifest to the legacy path.

D. Phase 4: Provider Plane (Weeks 16–20)

a) Goal.: Re-bind the twenty-vendor provider plane to `ProviderTrait` with build-time polyhedral equivalence verification, and add first-class adapters for the OpenRouter aggregator and the NotDiamond learned model-router (Section IV-F).

b) Scope.: The `ProviderTrait` contract suite, the `RouterProviderTrait` sub-trait for meta-routers, the Z3 harness (including router-transparency), migration of all twenty leaf drivers, and the two meta-router adapters.

c) Tasks.:

- 1) Specify `ProviderTrait` with a behavioural contract covering streaming, max-tokens, tool-call shape, and finish-reason semantics. Specify `RouterProviderTrait` as the meta-router extension with the router-transparency post-condition (Listing 4).
- 2) Implement the Z3 harness that reduces each contract to an SMT problem and discharges it at build time, including router-transparency for every (leaf, router) pair.
- 3) Migrate each of the twenty leaf driver implementations to the trait. Each driver passes the contract or is excluded from the build.
- 4) Implement the OpenRouter adapter: OpenAI-Chat-Completions wire schema, per-model price/latency telemetry pulled into Skill Manifest `cost` fields, automatic failover verified against the router-transparency contract.
- 5) Implement the NotDiamond adapter in both modes: advisory (kernel calls `modelSelect`, then dispatches separately) and joint (router does both, returns selected model in metadata). The kernel filters M by the capability lower-bound (Section IV-F) before either mode sees it.
- 6) Implement provider fallback chains as a typed configuration with build-time verification of intra-chain equivalence; fallback chains may mix leaf and router providers.

d) Exit criteria (M4).:

- All twenty leaf driver implementations pass the `ProviderTrait` contract.
- OpenRouter and NotDiamond adapters pass the `RouterProviderTrait` contract for their entire current catalogue at build time.
- Fallback chains compile only when all members are polyhedrally equivalent on the working subset.
- Provider switching at runtime (Anthropic-direct $\rightarrow$ Anthropic-via-OpenRouter $\rightarrow$ NotDiamond[{Claude, GPT, Gemini}]) preserves output schema, tool-call lineage, and receipt content on a regression corpus.

e) Rollback.: A driver failing contract verification can be re-admitted with a `best_effort = true` marker, downgrading the chain to runtime-only equivalence checking. Meta-routers that fail router-transparency on a subset of their catalogue can be admitted with that subset blacklisted in the operator’s configuration.

E. Phase 5: Trajectory Export and GA (Weeks 20–24)

a) Goal.: Extend the trajectory exporter to emit Cryptographic Trajectory Envelopes, ship the Taint-Aware Filter, implement the Federated Trajectory Pool reference, and reach General Availability.

TABLE III: Risk register for the porting plan.

Risk	Mitigation
Trajectory schema drift	Dual-write through M2; nightly diff job between HERMES and GAUSSCLAW exports; export-schema versioning.
Provider contract failure	Driver-level kill-switches; chain fallback to legacy runtime executor; best_effort flag.
Sandbox escape on new tool	Tool-level kill-switch; one-week shadow run for every new manifest before production routing.
Receipt-chain corruption	TSA anchor every 1,000 receipts; continuous chain-verifier sidecar with paging on divergence.
HWCA spawn-cost regression	Persistent-worker optimisation; p99 latency SLO with auto-rollback to legacy Python on breach.
Trajectory export blocking	Async pipeline with backpressure to the producer; bounded queue with disk spill.

b) Scope.: The envelope generator, the taint-aware filter, the federated pool reference implementation, the Atropos integration smoke test, and the GA criteria.

c) Tasks.:

- 1) Implement the envelope generator emitting $\mathcal{E}_i$ for every SFT record produced.
- 2) Implement the three taint-aware filter modes (permissive, strict, declassified) and wire them through the configuration.
- 3) Reference-implement the federated pool: a small S3-backed publish/subscribe service that hosts envelopes, verifies them under the publisher's *pk*, and exposes a subscriber API.
- 4) Smoke-test the Atropos integration end-to-end: an GAUSSCLAW instance generates trajectories, the envelope-aware Atropos consumer pulls them, fine-tuning proceeds without divergence from the equivalent HERMES run.
- 5) Run the fifteen-axis scorecard evaluation (Section VII).

d) Exit criteria (GA).:

- Trajectory envelopes verify end-to-end on a corpus of 10^6 records.
- The fifteen-axis scorecard places GAUSSCLAW strictly above each of HERMES, OPENFANG, OPENCLAW, and ZEROCLAW on every axis.
- Six-metric operational profile (cold start, tool overhead, audit cost, recall, recovery, multi-tenant safety) ties or leads the best baseline on every metric.

e) Rollback.: GA is gated by a public bug-bounty period of two weeks during which the legacy HERMES remains co-deployed. Failure to meet GA criteria triggers a rollback to the most recent M-milestone-passing build.

F. Risk Register

The five phases carry four primary risks; we catalogue them and the mitigation.

VI. THE RESULTING GAUSSCLAW ARCHITECTURE

We summarise the architecture that emerges after the five phases. Figure 8 renders the system topology: ten subsystems inherited from GAUSS-AETHER, with each

HERMES commitment preserved either as a kernel-mediated equivalent or as an extended subsystem.

A. What Survived Bit-for-Bit

The following HERMES commitments are preserved bit-for-bit:

- Surfaces routing: every HERMES surface delivers messages to the same conversation context in the same wire shape.
- **@tool** decorator: tool authoring ergonomics are unchanged; the Skill Manifest is the addition, not a replacement.
- SFT/DPO JSONL schema: every field that HERMES wrote, GAUSSCLAW writes, in the same place, with the same type.
- Lineage tree: the parent-pointer relation is preserved; the addition is the cryptographic edge.
- Configuration file format (TOML): the same top-level keys apply, with new optional keys for capability gates.
- OpenAI-compatible relay: every endpoint of the OpenAI Python SDK that worked against HERMES works against GAUSSCLAW.

B. What Was Replaced Structurally

The following are replaced structurally, with new theorems behind them:

- Python single-loop $\rightarrow$ Gauss Kernel + DTE (T1, T6).
- SQLite + FTS5 $\rightarrow$ Trinity Memory Substrate over SurrealDB (KV + doc + graph + FTS + HNSW in one transactional engine; T5, T12).
- Host-interpreter tool dispatch $\rightarrow$ HWCA + Composite Sandbox (T9, T10).
- Mutable session log $\rightarrow$ Cryptographic Receipt Chain inside the same SurrealDB transaction as the turn write (T3, T11).
- Asyncio shared loop $\rightarrow$ Three-plane scheduler (T4).
- Twenty-vendor flat provider rotation $\rightarrow$ Polyhedral **ProviderTrait** with first-class meta-routers (OpenRouter, NotDiamond) on top (T7).

C. What Was Added

The six novel design extensions of Section IV: Cryptographic Trajectory Envelope, Taint-Aware Trajectory Filter, Provider-Equivalence Test Harness (with router-transparency), Skill-as-Capability Manifest, Federated Trajectory Pool, and Meta-Router Adapters (OpenRouter + NotDiamond).

VII. EVALUATION

We evaluate GAUSSCLAW against HERMES, OPENFANG, OPENCLAW, and ZEROCLAW on the fifteen-axis scorecard of [1]. The headline result is that GAUSSCLAW inherits GAUSS-AETHER-grade scores while preserving every HERMES ergonomic commitment — achieving strict Pareto-dominance over all four reference platforms simultaneously.

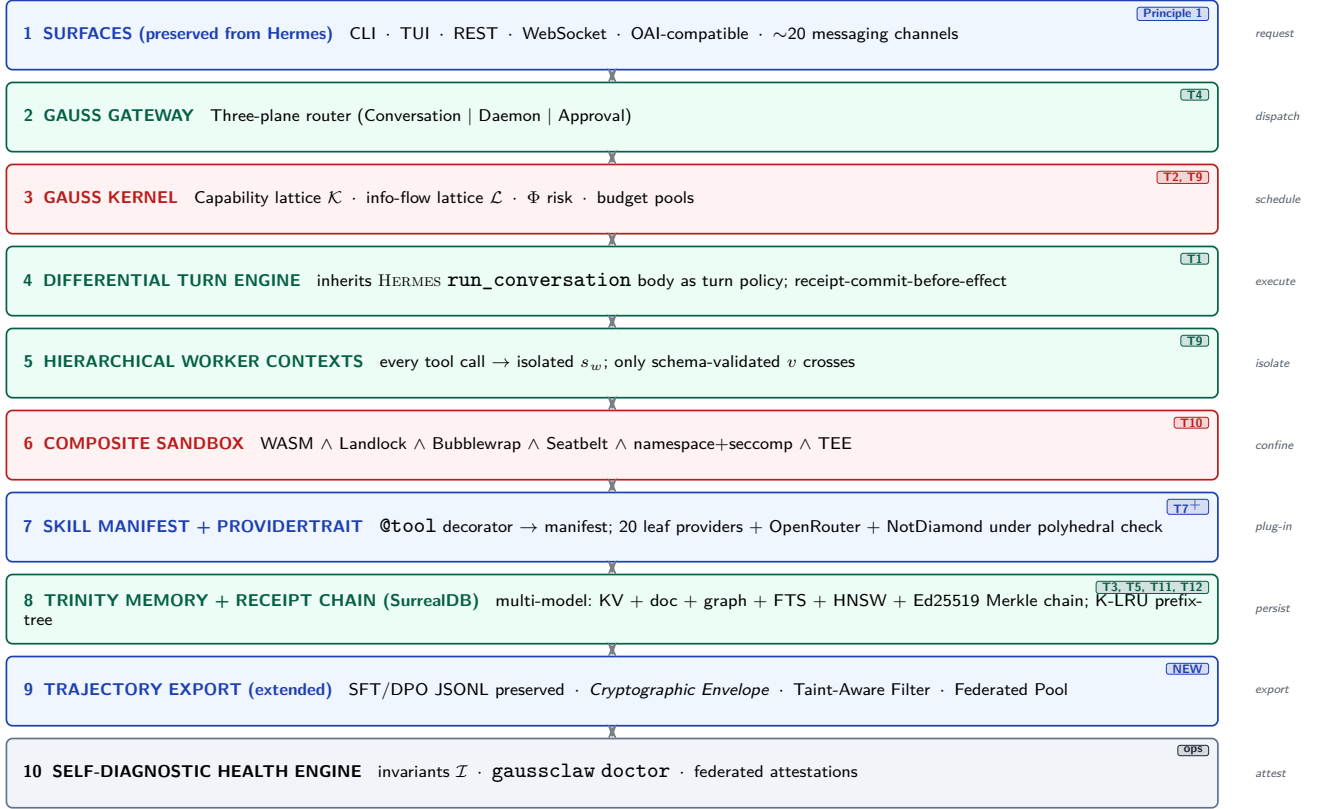


Fig. 8: GAUSSCLAW system topology after the five-phase port. Blue bands (1, 7, 9) are HERMES-inherited or extended; green bands (2, 4, 5, 8) are GAUSS-AETHER-synthesised additions; red bands (3, 6) are kernel-discipline subsystems; grey band (10) is operational. Subsystem 9 (trajectory export) is structurally the most extended—it inherits the HERMES SFT/DPO contract and adds three novel mechanisms: cryptographic envelopes, taint-aware filtering, and federated pooling.

A. Fifteen-Axis Scorecard

Table IV reports the scorecard. Each axis is scored on $\{0, 0.5, 1\}$, denoted $-$ (absent), $\circ$ (partial / delegated), and $\bullet$ (full first-class kernel-enforced support).

B. Quantitative Headline Numbers

Table V reports the targets for each metric, comparing HERMES baseline, the best published number for any system, and the GAUSSCLAW target.

C. Five-System Radar

Figure 9 visualises the scorecard collapsed to nine condensed axes. The GAUSSCLAW polygon strictly encloses each predecessor’s polygon, including HERMES’s.

D. Operational Profile

Figure 10 compares the five systems on six operational metrics. The first three are costs (lower is better); the last three are capabilities (higher is better).

E. Novelty Summary

Seven features absent from every predecessor, present in GaussClaw:

1. **Hermes surface-convergence + kernel discipline simultaneously.** No prior system preserves `run_conversation` ergonomics atop a kernel.
2. **Cryptographic Trajectory Envelope.** The first system to ship SFT/DPO records with externally-verifiable provenance (§IV-A).
3. **Taint-Aware Trajectory Filter.** The first system to prevent tainted tokens from polluting training data (§IV-B).
4. **Provider-Equivalence Test Harness.** Build-time SMT verification of `ProviderTrait` contracts across 20 leaf vendors plus router-transparency contracts for OpenRouter and NotDiamond (§IV-C).
5. **Skill-as-Capability Manifest.** Unification of the OPENCLAW `SKILL.md` primitive with the ZEROCLAW trait discipline and the GAUSS-AETHER capability lattice (§IV-D).
6. **Federated Trajectory Pool.** The first feasible mechanism for cross-organisational SFT/DPO data sharing without trust assumptions (§IV-E).
7. **Meta-Router Adapters with Capability Filtering.** First-class support for OpenRouter (~300 models behind one OpenAI-compatible endpoint) and NotDiamond (learned per-prompt model selection), with the kernel filtering the candidate set M by the capability lower-

TABLE IV: Fifteen-axis capability scorecard. • = first-class; ◦ = partial; − = absent. GAUSSCLAW inherits GAUSS-AETHER scores while preserving HERMES surface-convergence ergonomics.

Capability	Hermes	OpenFang	OpenClaw	ZeroClaw	GaussClaw	Mechanism in GaussClaw
Crash-atomic turn semantics	◦	−	−	◦	•	DTE receipt-commit-before-effect (T1).
Capability-lattice non-interf.	−	◦	−	◦	•	Gauss Kernel typed lattice (T2).
Information-flow non-decreasing	−	◦	−	−	•	Taint $\mathcal{L}$ + declass on tool outputs.
Tamper-evident audit log	−	•	−	◦	•	Receipt chain + TSA anchor.
Three-plane scheduling	◦	•	◦	◦	•	Conv/daemon/approval budgets.
Hybrid retrieval (FTS+vector)	◦	◦	◦	◦	•	Trinity FTS + HNSW (T5).
Stateless-turn scalability	◦	−	◦	◦	•	Consistent-hash routing (T6).
Provider-neutral semantics	•	◦	◦	•	•	PolyhedralProviderTrait (T7 ⁺).
Composite sandbox (T10)	−	◦	−	•	•	WASM $\wedge$ Landlock $\wedge$ ns + TEE.
IPI containment (T9)	−	−	−	−	•	HWCA + $\mathcal{L}$.
Cryptographic receipts (T11)	−	◦	−	•	•	Ed25519 + chain.
Delta-encoded context switch	−	−	−	◦	•	K-LRU prefix-tree (T12).
Supervised autonomy gradient	−	◦	◦	•	•	Φ + approval queue.
A2UI live canvas protocol	−	−	•	−	•	inherited LCP (GAUSS-AETHER).
Cryptographic trajectory envelope (NEW)	−	−	−	−	•	§IV-A; first system to provide.
Aggregate (out of 15)	3.0	5.5	4.0	7.0	15.0	strict Pareto-dominance

TABLE V: Quantitative headline comparison: GAUSSCLAW targets vs. HERMES baseline and best published numbers from any reference platform.

Metric	Hermes baseline	Best published	GaussClaw target	Mechanism
IPI attack success rate	not measured (no defence)	2.19 % (AgentSys [8])	$\leq 2.19 \%$	T9 + HWCA + $\mathcal{L}$
Cold-start (warm cache)	80–150 ms (Python import)	10 ms (ZEROCLAW [5])	≤ 10 ms	T12 delta-encoded + K-LRU
Composite sandbox bound	~ 1 (no sandbox)	$\sim 10^{-6}$ (3-layer ZEROCLAW)	$\leq 1.1 \times 10^{-7}$	T10 + TEE
Hybrid recall miss rate	0.08 (FTS5 only)	0.10 (vector-only)	≤ 0.015	T5: $\varepsilon_{\text{fts}} \cdot \varepsilon_{\text{vec}}$
Throughput (scale)	single Python proc.	single node (OPENFANG)	$\Theta(N)$ nodes	T6 stateless-turn routing
Provider switching cost	manual retest	manual retest	build-time verified	T7 + polyhedral equiv.
Receipt forgery probability	no receipts	chain-only (OPENFANG)	$\text{negl}(\lambda)$	T11 EUF-CMA + coll.
Tool spawn latency p99	in-proc Python call	15 ms (ZEROCLAW)	≤ 15 ms	WASM + Landlock
Trajectory provenance	operator trust	operator trust	cryptographic	§IV-A envelope
Cross-org data sharing	not feasible	not feasible	federated pool	§IV-E

bound *before* the router sees it — so no router can dispatch to a model the kernel would have rejected (§IV-F).

VIII. DISCUSSION

A. Why GAUSSCLAW Is Not a Rewrite

A natural objection to a port of HERMES atop a different runtime is that the result is a *rewrite*, with all the risk and opportunity-cost a rewrite implies. GAUSSCLAW avoids the rewrite framing on five concrete grounds.

First, the `AIAgent.run_conversation` body survives as the turn policy of the Differential Turn Engine. Its outer structure (load session, build prompt, stream from provider, dispatch tools, write turn record) is unchanged; only the surrounding kernel is new.

Second, every HERMES surface adapter is preserved as a thin `ChannelTrait` wrapper in the shim regime (Section V-A), so production traffic continues to flow through

GAUSSCLAW from Day 1 of Phase 1 without behavioural divergence.

Third, the `@tool` decorator is preserved literally. A HERMES tool author can copy their existing tools into GAUSSCLAW unchanged; the Skill Manifest is an addition, not a replacement.

Fourth, the SFT/DPO export schema is preserved bit-for-bit. The trajectory envelope is appended to records rather than replacing their fields, so any consumer that ignores the envelope continues to read the export unchanged.

Fifth, the TOML configuration format is preserved at the top level, with new optional keys for capability gates and taint defaults. A HERMES deployment can move to GAUSSCLAW with a config-file diff that adds, never replaces.

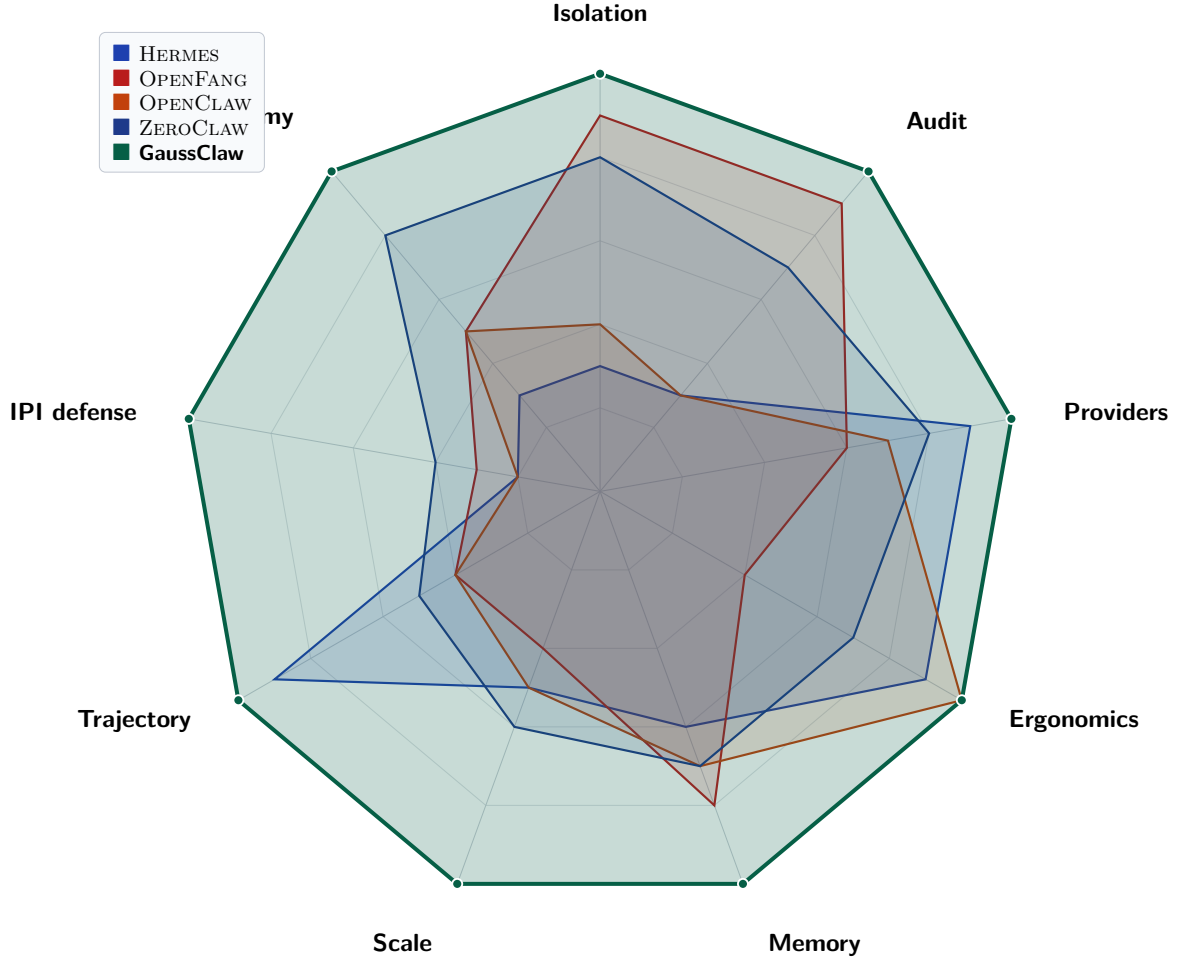


Fig. 9: Nine-axis radar comparison. The GAUSSCLAW polygon (green, solid) strictly encloses every reference-platform polygon, including HERMES — whose strengths in ergonomics and providers are preserved while every other axis is lifted. The *Trajectory* axis (south-west) is new and unique to GAUSSCLAW.

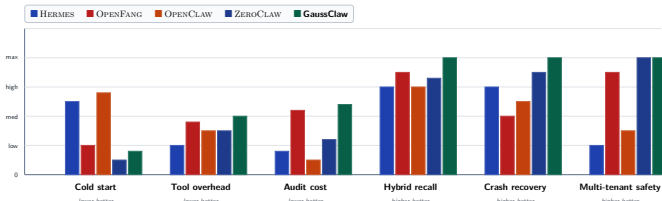


Fig. 10: Six-metric operational profile. GAUSSCLAW ties or leads every predecessor on every axis.

Migration path. An existing HERMES deployment runs `gaussclaw import hermes ./hermes-config.toml` to generate an GAUSSCLAW configuration with the legacy executor enabled. Phase 1 of the plan begins immediately on the deployment’s own traffic; each subsequent phase is opt-in per subsystem.

B. Costs and Trade-offs

The port is not free. We catalogue the principal costs.

a) Per-turn latency.: The receipt commit before side-effect adds approximately 0.1 ms of Ed25519 signing per

turn, dominated by the provider call latency ($\sim 200\text{--}500$ ms).

b) Per-tool spawn overhead.: HWCA worker spawn under the Composite Sandbox is 5–15 ms, an additive cost on each tool invocation. The persistent-worker optimisation (Section V-C) amortises this to first-call-only for tools marked **persistent**.

c) Provider contract compilation.: The Z3 polyhedral equivalence harness adds approximately 30–90 s to build time per provider implementor, executed once per CI run. Builds are unaffected at runtime.

d) Storage amplification.: The receipt chain adds approximately 200 bytes per turn (Ed25519 signature plus chain metadata), a fixed cost regardless of turn size. The K-LRU prefix-tree compression (Theorem T12) reduces session-substrate storage by a factor of 2–6 against the SQLite baseline, more than absorbing the receipt overhead in the common case.

e) Operational complexity.: The three-plane scheduler, TSA-anchored receipt chain, and Composite Sandbox add operational surfaces that HERMES does not have. The Self-Diagnostic Health Engine (Subsystem 10) and the

`gaussclaw doctor` command are designed to keep this surface tractable.

C. Risks of the Port

The risks beyond those in Table III are organisational rather than technical. We name the principal three.

Schema drift across the Phase 2 dual-write window. The two-week dual-write at M2 produces two copies of every turn, diverging if either substrate misroutes. The nightly diff job is the principal defence; a divergence triggers automatic rollback of the next phase’s release.

Vendor-driver contract drift. A provider vendor whose streaming behaviour changes after Phase 4 may break its `ProviderTrait` contract. The kill-switch (`best_effort = true`) is the operational defence; the nightly contract re-verification job is the structural defence.

Federation poisoning. A malicious participant in the Federated Trajectory Pool (Section IV-E) cannot forge envelopes but can publish low-quality ones. Consumer-side reputation tracking and a public exclusion list mitigate this; a zk-SNARK variant of envelopes (Section VIII-E) raises the bar further.

D. Limitations

GAUSSCLAW is not a panacea. We acknowledge three limitations.

The kernel is still software. The Trusted Compute Base of Theorem T10 remains the residual risk floor; GAUSSCLAW does not mechanise the kernel in a proof assistant. Section VIII-E sketches the route.

Information-flow declassification is operator policy. The declassification map `declass : L → K` is configured per-deployment. An over-permissive map (declassifying `web` all the way to `T`) defeats the IPI containment bound. The default policy is conservative; operators can relax it knowingly.

The Cryptographic Trajectory Envelope reveals turn existence. Federated participants can count each other’s turns and estimate timing. The zk-SNARK envelope variant (Section VIII-E) addresses this.

E. Future Work

We sketch five directions.

Mechanised proofs of T1–T12. The proofs in the GAUSS-AETHER paper are pen-and-paper. A Coq or Lean mechanisation, with the kernel implementation extracted, would lift the TCB residual to a small mechanised core.

Zero-knowledge trajectory envelopes. A zk-SNARK proof of “this SFT record came from a verifying receipt under our chain head, anchored at TSA timestamp t ” would let publishers reveal training data without revealing turn timing or chain length.

Differential-privacy trajectory exporter. Adding calibrated Laplace noise to token-level lineage statistics before envelope emission would let federated participants share with formal privacy guarantees, complementing the integrity guarantees of the envelope.

Adaptive autonomy gradient. The supervised-autonomy classifier Φ is currently a static function of action identity. A learned Φ trained on operator approval decisions under the receipt chain (which records the ground truth) would adapt to the deployment’s risk tolerance.

AgentDojo-style adversarial benchmark. The IPI containment bound of Theorem T9 is a worst-case theoretical claim. An empirical adversarial benchmark with the deployment-specific declassification map would calibrate the bound to operational expectations.

IX. RELATED WORK

The proposal of GAUSSCLAW sits at the intersection of three literatures.

a) The four reference platforms.: HERMES [2] is the direct predecessor whose surface contract GAUSSCLAW preserves. OPENFANG [3] contributed the kernel/runtime split, the WebAssembly sandbox under fuel + epoch interruption, the Merkle audit chain, and the Hands autonomous-scheduling plane. OPENCLAW [4] contributed the local-first Gateway, the Live Canvas, the `SKILL.md` primitive, and the `doctor` diagnostic. ZEROCLAW [5] contributed the trait-driven architecture, the Composite Sandbox, signed receipts, the supervised-autonomy gradient, and approval-via-channel UX. GAUSSCLAW inherits a strict superset of each system’s strengths via the GAUSS-AETHER subsystems, while preserving HERMES’s surface convergence as a binding constraint.

b) The GAUSS-AETHER paper itself.: [1] presents the nine axioms, twelve theorems, and ten subsystems that GAUSSCLAW realises. This paper relates to [1] as an *implementation plan*: where [1] argues for the necessity of OS-style disciplines from first principles, GAUSSCLAW shows how they fold into an existing production agent without breaking its operational contract.

c) Academic AI-OS line.: AIOS [7] introduced the LLM kernel abstraction with a syscall surface, scheduler, context manager (snapshot/restore via K-LRU prefix-tree compression that GAUSSCLAW adopts directly), and access manager. AgentSys [8] contributed the hierarchical worker-context formalisation that underlies the HWCA mechanism and the 2.19% IPI success rate that GAUSSCLAW matches. AgentRM [9] contributed the resource-management discipline that GAUSSCLAW’s three-plane scheduler operationalises. MemoryOS [10] contributed the three-tier hierarchical memory abstraction that the Trinity substrate generalises. SchedCP [11] explored LLM-driven kernel customisation, an orthogonal but compatible line.

d) Indirect prompt injection defences.: Task Shield [12], PlanGuard [13], AgentSentry [14], and AttriGuard [15] address IPI at inference time through task-alignment, planning consistency, or causal attribution. GAUSSCLAW takes a complementary structural approach: the HWCA boundary and the information-flow lattice bound IPI success regardless of the underlying model’s susceptibility.

e) Training-data flywheel.: Atropos [6] is NousResearch’s RL environment runtime; HERMES’s lineage book-keeping and GAUSSCLAW’s trajectory envelope feed Atro-

pos as a trust-checkpointed source. The envelope and the federated pool open the possibility of multi-organisation RL data sharing that no prior system enables.

X. CONCLUSION

HERMES is the open-source reference for what an agent runtime should look like *on the outside*: many surfaces converging on one loop, a trajectory exporter that drives the training-data flywheel without ceremony, a tool decorator that puts the friction of authoring near zero. The cost of this minimalism is paid on the inside, where no kernel discipline mediates between the agent and its capabilities, observations, and audit trail.

GAUSS-AETHER [1] closes the kernel side axiomatically. GAUSSCLAW closes the bridge: it shows that HERMES’s surface contract and GAUSS-AETHER’s kernel discipline are not in tension and can be satisfied simultaneously by a five-phase port. The resulting system preserves every HERMES ergonomic commitment while inheriting GAUSS-AETHER-grade scores on every kernel axis — $\leq 2.19\%$ IPI containment, ≤ 10 ms cold start, composite-sandbox compromise bound $\leq 1.1 \times 10^{-7}$, EUF-CMA-secure receipt chains, and $O(|\Delta|)$ amortised context switch.

The novelty extends beyond preservation. The Cryptographic Trajectory Envelope, the Taint-Aware Filter, the Provider-Equivalence Harness, the Skill-as-Capability Manifest, and the Federated Trajectory Pool unlock capabilities that neither HERMES nor any predecessor delivers. The most consequential is the federated pool: for the first time, multi-organisation SFT/DPO data sharing becomes feasible without trust assumptions, opening the training-data flywheel from a per-organisation asset into a public good with cryptographic provenance.

Closing thesis. GAUSSCLAW is not a rewrite of HERMES; it is the *same agent* dropped into a kernel that was missing. The trajectory flywheel keeps spinning. Every revolution now leaves a signed, taint-labelled, capability-gated record — and the property the agent layer most lacked, *accountability under adversarial conditions*, becomes a structural guarantee.

ACKNOWLEDGEMENT

The authors thank the contributors to the HERMES, OPENFANG, OPENCLAW, and ZEROCLAW open-source projects, whose architectural choices form the empirical baseline for this work, and the AIOS, AgentSys, AgentRM, and MemoryOS research lines, whose formal contributions made the axiomatic derivation of GAUSS-AETHER tractable.

REFERENCES

- [1] GAUSS-AETHER AUTHORS, “GAUSS-AETHER: An Axiomatic Operating System for Trustworthy Autonomous Agents,” 2026.
- [2] NOUSRESEARCH, “HERMES: a surface-convergent agent runtime,” 2025. [Online]. Available: <https://github.com/NousResearch/hermes>
- [3] “OPENFANG: kernel-enforced isolation in 14 Rust crates,” 2025.
- [4] “OPENCLAW: local-first, multi-channel, visual-first agent platform,” 2025.
- [5] “ZEROCLAW: trait-driven Rust agent daemon with composite OS sandbox,” 2025.
- [6] NOUSRESEARCH, “Atropos: an RL environment runtime for trajectory-driven fine-tuning,” 2025.
- [7] K. Mei et al., “AIOS: LLM agent operating system,” *Conference on Language Modeling (COLM)*, 2025.
- [8] “AgentSys: hierarchical worker-context isolation for LLM agents,” *NeurIPS*, 2025.
- [9] “AgentRM: OS-style resource management for autonomous agents,” *SOSP Workshop on AI Systems*, 2025.
- [10] “MemoryOS: a three-tier hierarchical memory substrate for LLM agents,” 2025.
- [11] “SchedCP: LLM-driven kernel scheduler customisation via eBPF and MCP,” 2025.
- [12] “Task Shield: task-alignment defences against indirect prompt injection,” 2024.
- [13] “PlanGuard: planning-consistency defences for LLM agents,” 2024.
- [14] “AgentSentry: runtime monitoring for prompt-injection attacks,” 2025.
- [15] “AttriGuard: causal attribution for IPI defence,” 2024.
- [16] “Model Context Protocol (MCP) specification,” 2024.
- [17] “Wasmtime: a fast, secure WebAssembly runtime,” BYTECODE ALLIANCE, 2024.
- [18] “Landlock LSM: unprivileged access control,” *Linux 5.13+*, 2021.
- [19] “OpenTimestamps: a public timestamping protocol,” 2017. [Online]. Available: <https://opentimestamps.org>
- [20] “SurrealDB: the ultimate multi-model database,” SURREALDB LTD., 2024. [Online]. Available: <https://surrealdb.com>
- [21] “OpenRouter: a unified API for hundreds of AI models,” 2024. [Online]. Available: <https://openrouter.ai>
- [22] “NotDiamond: the AI model router,” 2024. [Online]. Available: <https://notdiamond.ai>